

UNIVERSITY COMPLAINT HELPER

A PROJECT REPORT

Submitted by

Aryan Mauryakant (24BAI10142)

Tanishq Ganesh Gawade (24BAI10253)

Aditya Aman (24BAI10129)

Raj Pratap Singh (24BAI10722)

Manthan Rohira (24BAI10670)

Naresh Suthar (24BAI10260)

*in partial fulfillment for the award of the degree
of*

BACHELOR OF TECHNOLOGY

**COMPUTER SCIENCE AND ENGINEERING
(ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING)**



**SCHOOL OF COMPUTING SCIENCE ENGINEERING AND ARTIFICIAL
INTELLIGENCE**

VIT BHOPAL UNIVERSITY

**KOTRIKALAN, SEHORE
MADHYA PRADESH – 466114**

SEPTEMBER 2025

**VIT BHOPAL UNIVERSITY, KOTRIKALAN, SEHORE
MADHYA PRADESH – 466114**

BONAFIDE CERTIFICATE

Certified that this project report titled “UNIVERSITY COMPLAINT HELPER ” is the bonafide work of “Naresh Suthar (24BAI10260) , Manthan Rohira (24BAI10670) , Raj Pratap Singh (24BAI10722) , Aditya Aman (24BAI10129) , Tanishq Ganesh Gawade (24BAI10253) , Aryan Mauryakant (24BAI10142)” who carried out the project work under my supervision. Certified further that to the best of my knowledge the work reported at this time does not form part of any other project/research work based on which a degree or award was conferred on an earlier occasion on this or any other candidate.

PROGRAM CHAIR

Dr. Siddharth Singh Chouhan
School of Computing Science Engineering
and Artificial Intelligence

VIT BHOPAL UNIVERSITY

PROJECT GUIDE

Dr. G Prabu Kanna
School of Computing Science Engineering
and Artificial Intelligence

VIT BHOPAL UNIVERSITY

The Project Exhibition I Examination is held on 25 September 2025.

ACKNOWLEDGEMENT

First and foremost I would like to thank the Lord Almighty for His presence and immense blessings throughout the project work.

I wish to express my heartfelt gratitude to Dr....., Head of the Department, SCAI for much of his valuable support encouragement in carrying out this work.

I would like to thank my internal guide Mr. G Prabu kanna ,for continually guiding and actively participating in my project, giving valuable suggestions to complete the project work.

I would like to thank all the technical and teaching staff of the School of Aeronautical Science, who extended directly or indirectly all support.

Last, but not least, I am deeply indebted to my parents who have been the greatest support while I worked day and night for the project to make it a success.

LIST OF ABBREVIATIONS

S. No.	Symbol/Abbreviation	Full Form/Meaning
1	NLP	Natural Language Processing
2	TF IDF	Term Frequency-Inverse Document Frequency
3	SVM	Support Vector Machine
4	ML	Machine Learning
5	CSV	Comma Separated Values
6	AI	Artificial Intelligence
7	GUI	Graphical User Interface
8	API	Application Programming Interface
9	JSON	JavaScript Object Notation
10	PKL	Pickle File Format
11	NLTK	Natural Language Toolkit
12	IDF	Inverse Document Frequency
13	BOW	Bag of Words

LIST OF FIGURES AND GRAPHS

FIGURE NO.	TITLE	PAGE NO.
1.	Software Architectural designs	29
2.	Student Interface	30

ABSTRACT

Purpose:

The proliferation of digital communication in educational institutions has led to an exponential increase in student complaints across various categories. Manual processing of these complaints is time-intensive, error-prone, and often leads to delayed responses, affecting student satisfaction and administrative efficiency. This project presents an automated University Complaint Classifier system that leverages Natural Language Processing (NLP) techniques to intelligently categorize and route student complaints to appropriate departments for swift resolution.

Methodology:

The proposed system employs a comprehensive NLP pipeline integrating text preprocessing, Term Frequency Inverse Document Frequency (TF-IDF) vectorization, and supervised machine learning classification. The system preprocesses complaint text through tokenization, stop-word removal, stemming, and normalization to prepare it for feature extraction. TF-IDF vectorization transforms the preprocessed text into numerical feature vectors that capture the semantic importance of words within the context of the entire complaint corpus. Multiple machine learning algorithms including Logistic Regression, Naive Bayes, Linear Support Vector Machine (SVM), and Random Forest were evaluated for classification performance. The Linear SVM classifier emerged as the optimal model based on accuracy, precision, recall, and F1-score metrics. The trained model is serialized using pickle format for deployment in the categorizer module, which provides real-time complaint classification and automated solution routing.

Findings:

Experimental evaluation demonstrates that the Linear SVM classifier achieves superior performance in complaint categorization across multiple categories including

Civil, Electrical, Plumbing, Air Conditioning, Lift, Water Geyser, Water Cooler, Carpentry, House Keeping, Mess, CTS Computing and Technical Services), COE (Controller of Examinations), DSW (Dean Student Welfare), Academics, and FFCS (Fully Flexible Credit System). The system achieves high accuracy with minimal latency, enabling real-time processing of incoming complaints. The automated categorizer not only classifies complaints but also provides standardized resolution pathways, guiding students through appropriate channels for complaint resolution. This dual functionality significantly reduces administrative overhead while ensuring consistent and timely responses to student concerns. The system's scalability and adaptability make it suitable for deployment across various educational institutions with minimal customization requirements.

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	List of Abbreviations	4
	List of Figures and Graphs	5
	Abstract	6,7
1	CHAPTER-1: PROJECT DESCRIPTION AND OUTLINE 1.1 Introduction 1.2 Motivation for the work 1.3 Project Concept and Techniques 1.4 Problem Statement 1.5 Objective of the work 1.6 Organization of the project 1.7 Summary	10
2	CHAPTER-2: RELATED WORK INVESTIGATION 2.1 Introduction 2.2 Existing Approaches/Methods 2.3 Analysis of existing methods 2.4 Issues/observations from investigation 2.5 Summary	16
3	CHAPTER-3: REQUIREMENT ARTIFACTS 3.1 Introduction 3.2 Hardware and Software requirements 3.3 Specific Project requirements 3.4 Summary	20
8 Page		

4	CHAPTER-4: DESIGN METHODOLOGY AND ITS NOVELTY 4.1 Methodology and goal 4.2 Functional modules design and analysis 4.3 Software Architectural designs 4.4 User Interface designs 4.5 Summary	25
5	CHAPTER-5: TECHNICAL IMPLEMENTATION & ANALYSIS 5.1 Outline 5.2 Technical coding and code solutions 5.3 Model Training and Evaluation 5.4 Categorizer Implementation 5.5 Performance Analysis 5.6 Summary	31
6	CHAPTER-6: PROJECT OUTCOME AND APPLICABILITY 6.1 Outline 6.2 key implementations Highlights 6.3 Significant project outcomes 6.4 Project applicability on Real-world applications 6.5 Inference	40
7	CHAPTER-7: CONCLUSIONS AND RECOMMENDATION 7.1 Outline 7.2 Limitation/Constraints of the System 7.3 Future Enhancements 7.4 Inference	44
	References	47

CHAPTER – 1

PROJECT DESCRIPTION AND OUTLINE

1.1 Introduction

The digital transformation of educational institutions has fundamentally altered the landscape of student administration interactions. Modern universities handle thousands of complaints daily across diverse categories ranging from infrastructure issues to academic concerns. Traditional manual processing of these complaints presents significant challenges including inconsistent categorization, delayed responses, and inefficient resource allocation.

The emergence of Natural Language Processing NLP and machine learning technologies offers unprecedented opportunities to automate and optimize complaint management systems. By leveraging computational linguistics and pattern recognition algorithms, educational institutions can develop intelligent systems that accurately classify complaints and provide standardized resolution pathways.

This project presents a comprehensive University Complaint Classifier system that employs advanced NLP techniques to automatically categorize student complaints and provide appropriate solution frameworks. The system integrates text preprocessing, feature extraction through TF IDF vectorization, and supervised machine learning classification to achieve high accuracy in complaint categorization across multiple domains including infrastructure, academics, administrative services, and student welfare.

The automated system not only reduces administrative workload but also ensures consistent and timely responses to student concerns, ultimately enhancing the overall educational experience. The implementation demonstrates the practical application of

artificial intelligence in educational administration, providing a scalable solution that can be adapted across various institutional contexts.

1.2 Motivation for the Work

The motivation for developing an automated complaint classification system stems from several critical challenges observed in contemporary university administration. Manual complaint processing suffers from inherent limitations including subjective interpretation, inconsistent categorization criteria, and scalability constraints. Administrative staff often struggle to maintain uniform standards when processing large volumes of complaints, leading to disparities in response quality and resolution timelines.

Furthermore, the increasing diversity of complaint types and the growing student population exacerbate these challenges. Universities must handle complaints spanning infrastructure maintenance, academic policies, examination procedures, hostel facilities, technical services, and student welfare programs. Each category requires specialized knowledge and specific resolution protocols, making manual classification increasingly complex and error-prone.

The advent of sophisticated NLP technologies and machine learning algorithms provides an opportunity to address these challenges systematically. By developing an intelligent classification system, universities can achieve consistent categorization standards, reduce processing times, and enable more efficient resource allocation. The system's ability to provide standardized solution pathways ensures that students receive appropriate guidance regardless of which administrative personnel initially handle their complaints.

1.3 Project Concept and Techniques

The University Complaint Classifier project uses an NLP pipeline to turn unstructured complaint text into actionable insights through systematic preprocessing, feature extraction, and machine learning. Text is cleaned with tokenization, stop-word removal, and stemming, then represented using TF-IDF, which captures both word importance and semantic significance. Multiple supervised algorithms, including Logistic Regression, Naive Bayes, Linear SVM, and Random Forest, are tested to select the best performer based on accuracy, efficiency, and generalization. The final model powers a deployment-ready categorizer that classifies complaints in real time and suggests automated solutions, with a modular design that ensures easy integration and scalability. The innovation lies in its domain-specific optimization, thorough model evaluation, and combined classification-solution approach.

1.4 Problem Statement

Contemporary university complaint management systems struggle with inefficiency, inconsistency, and lack of scalability due to manual processing. Staff must individually read and categorize each complaint, leading to subjective interpretation, delays in response, and inconsistent outcomes. The manual approach also creates scalability issues as student numbers and complaint volumes increase, causing rushed analyses and suboptimal resolutions. Without automation, universities lack systematic data collection, preventing them from identifying recurring issues or improving processes effectively. This highlights the urgent need for an automated classification system that ensures accurate categorization, standardized responses, faster resolution, and actionable insights for institutional advancement.

1.5 Objective of The work

The project's objectives are designed to overcome key challenges in university complaint management using NLP and machine learning, focusing on both technical advancements and practical benefits.

Technical Objectives

- Build a robust NLP preprocessing pipeline with tokenization, normalization, stop-word removal, and stemming.
- Apply TF-IDF vectorization to convert text into meaningful numerical features.
- Evaluate Logistic Regression, Naive Bayes, Linear SVM, and Random Forest to identify the best classifier.
- Develop an automated real-time categorizer that classifies complaints and suggests solutions without human input.

Practical Objectives

- Achieve high classification accuracy for reliable complaint routing.
- Minimize processing time through near-instant automated classification.
- Ensure consistent categorization by removing human subjectivity.
- Provide clear solution pathways tailored to complaint categories.

Secondary Objectives

- Showcase real-world application of NLP in education administration.

- Create an adaptable framework usable in other institutions.
- Establish baseline metrics for enhancement and performance evaluation.

1.6 Organization of the project

This project report is systematically organized into seven comprehensive chapters that provide detailed coverage of all aspects of the University Complaint Classifier development and implementation. Each chapter builds upon previous sections while maintaining clear focus on specific project components.

Chapter 1 presents the foundational understanding of the project including the introduction to automated complaint classification, motivation for developing the system, technical concepts and methodologies employed, problem statement definition, and comprehensive project objectives. This chapter establishes the contextual framework for the entire project.

Chapter 2 provides extensive investigation of related work in the domains of complaint management systems, text classification methodologies, and NLP applications in educational contexts. This chapter analyzes existing solutions, identifies their limitations, and positions the current project within the broader research landscape

Chapter 3 details the comprehensive requirement analysis including hardware and software specifications, functional requirements, performance criteria, and system constraints. This chapter provides the technical foundation necessary for implementation planning and system design.

Chapter 4 presents the detailed design methodology including system architecture, functional module specifications, algorithmic approaches, and user interface

considerations. This chapter demonstrates the systematic approach employed in developing an effective and scalable solution.

Chapter 5 focuses on technical implementation including code development, model training procedures, performance evaluation, and system testing. This chapter provides comprehensive coverage of the development process and validates the technical approach through empirical results.

Chapter 6 analyzes project outcomes including key achievements, performance metrics, real-world applicability, and potential impact on university administration. This chapter demonstrates the practical value and broader implications of the implemented system.

Chapter 7 concludes the report with comprehensive analysis of system limitations, recommendations for future enhancements, and overall project assessment. This chapter provides strategic guidance for continued development and implementation.

1.7 Summary

This introductory chapter outlines the framework of the University Complaint Classifier project by addressing the major challenges in manual complaint management and showing how NLP and machine learning can solve them effectively. It identifies key issues such as inconsistent categorization, delays, scalability limitations, and lack of standardized responses, all of which hinder efficiency and student satisfaction. The proposed technical approach integrates text preprocessing, TF-IDF vectorization, and supervised machine learning to ensure accurate, efficient classification, supported by thorough algorithm evaluation for optimal performance in real-world settings. Clearly defined objectives set measurable goals for both technical and practical impact, while the report's structured organization ensures clarity, logical flow, and comprehensive coverage of technical details and results.

CHAPTER – 2

RELATED WORK INVESTIGATION

2.1 Introduction

This chapter reviews the intersection of Natural Language Processing and complaint management, focusing on automated classification, text mining, and machine learning in educational administration. It analyzes traditional rule-based and modern machine learning methods, assessing their effectiveness and limitations in university complaint management. The review identifies research gaps and situates the current project within the broader field of automated text classification. It covers key aspects such as preprocessing, feature extraction, classification algorithms, evaluation metrics, and deployment, with emphasis on domain-specific challenges in educational contexts.

2.2 Existing Approaches/Methods

Traditional complaint management systems relied on keyword matching and rule-based methods using predefined dictionaries. While efficient and interpretable, they struggled with new vocabulary, complex language, ambiguous cases, and required frequent updates. Research by Saha et al. showed only moderate accuracy in clear cases but poor handling of mixed or unclear complaints.

Machine learning improved performance by better handling variability and semantics. Bozyiğit et al. used SVM, Naive Bayes, and XGBoost for food industry complaints with notable gains over rule-based systems. In healthcare, Li et al. achieved 94% accuracy using TF-IDF with SVM for classifying patient issues like communication, diagnosis, and management problems.

Deep learning further advanced complaint classification with transformer models like BERT. Vinayak and Chandrasekharan found DistilBERT with CNNs reached a 93% F1-score, outperforming TF-IDF methods. However, deep learning demands large datasets, high computing power, and suffers from interpretability challenges, making it less practical for smaller institutions.

2.3 Analysis of existing methods

Text preprocessing is essential for accurate complaint classification, involving tokenization, stop-word removal, stemming/lemmatization, and normalization. Verma et al. found that well-preprocessed text boosts accuracy by 15–20%, with domain-specific customization often required.

TF-IDF is the most widely used feature extraction method, outperforming bag-of-words and competing well with embeddings. Incorporating bigrams and trigrams improves classification by 8–12% in academic and technical complaints.

Among algorithms, Support Vector Machines deliver the best balance of accuracy, efficiency, and interpretability with TF-IDF features. Naive Bayes is efficient but weaker on complex text, while Logistic Regression is interpretable but less effective with large vocabularies. Random Forests handle interactions well but risk overfitting on small datasets. Ensembles improve results yet add complexity.

Evaluation typically uses accuracy, precision, recall, and F1-score with cross-validation, and stratified sampling to address class imbalance. In educational contexts, metrics also emphasize routing accuracy, processing speed, and complaint resolution efficiency.

2.4 Issues/observations from investigation

Scalability is a major challenge, as most systems are tested on small datasets, while universities must process thousands of complaints across many overlapping categories. Performance often drops as vocabulary and category counts grow, and real-time response demands strain resource-limited institutions.

Domain adaptation is critical since general models lose 20–30% accuracy when applied to university complaints without customization. Student-specific vocabulary, informal language, multilingual inputs, and unique resolution processes require tailored preprocessing and feature extraction.

Integration adds further complexity, as systems must connect with existing portals and workflows while supporting continuous updates for new complaints. Yet, most research overlooks long-term maintenance and system architecture.

Performance consistency also remains problematic, with results varying due to dataset differences and lack of standardized benchmarks. This highlights the need for rigorous evaluation methods and common frameworks for meaningful comparison.

2.5 Summary

The investigation of related work shows that automated text classification is a mature field with strong potential for complaint management. Traditional rule-based methods offer efficiency and interpretability, but machine learning delivers higher accuracy and better handling of complex language. Among approaches, TF-IDF vectorization with Support Vector Machine classification stands out for balancing performance, interpretability, and computational cost, with proven success in healthcare, customer service, and initial educational settings. However, gaps remain in adapting methods to university-specific needs, scaling to large datasets, and integrating with institutional

systems. Educational complaint management requires tailored solutions to address diverse categories, complex cases, and real-time processing demands. This project tackles these gaps by creating a university-optimized system with multi-algorithm evaluation and deployment architecture, contributing both practical solutions for institutions and broader research value through detailed analysis and open-source implementation.

CHAPTER – 3

REQUIREMENT ARTIFACTS

3.1 Introduction

The successful development and deployment of the University Complaint Classifier system depends on clearly defined functional, non-functional, and technical requirements. This chapter outlines a detailed requirement analysis covering hardware specifications, software dependencies, performance standards, and operational constraints. The analysis was conducted through systematic consultations with administrators, IT staff, and student representatives to capture both user needs and institutional limitations. The resulting specifications guide system design, implementation, and evaluation, while also defining measurable success criteria. Requirements are categorized into hardware infrastructure, software dependencies, functional features, performance benchmarks, security protocols, and maintainability standards, with each category including specific metrics and validation methods to ensure compliance throughout development and testing.

3.2 Hardware and Software requirements\

Hardware Requirements :

- **Processor:** Intel Core i5 or AMD Ryzen 5
- **Memory:** 8 GB RAM for development and testing environments
- **Storage:** 50 GB available disk space

Software Requirements :

-Operating System:

Primary Support: Linux distributions Ubuntu 18.04

Secondary Support: Windows 10/11 Professional, macOS 10.14

-Python Environment:

Version: Python 3.7 or newer Python 3.8+ recommended)

Package Manager: pip 20.0+ or conda for dependency management

Virtual Environment: venv

-Core Dependencies:

pandas: 1.3.0+ for data manipulation and CSV processing

scikit-learn: 1.0.0+ for machine learning algorithms and preprocessing

nltk: 3.6+ for natural language processing utilities

joblib: 1.0.0+ for model serialization and parallel processing

numpy: 1.20.0+ for numerical computations

-Development Tools:

IDE PyCharm Professional, Visual Studio Code, or Jupyter Notebook

Version Control: Git 2.20+ for source code management

Testing Framework: pytest 6.0+ for unit and integration testing

3.3 Specific Project requirements

Functional Requirements

FR1: Text Preprocessing

- Implement tokenization, normalization, stop-word removal, and stemming.
- Handle multiple encodings, special characters, and formatting issues.

FR2: Feature Extraction

- Use TF-IDF with configurable parameters (max features 5000+, n-gram range 1–2).
- Support vocabulary filtering and output sparse matrices for ML models.

FR3: Model Training

- Support Logistic Regression, Naive Bayes, Linear SVM, and Random Forest.
- Evaluate models using accuracy, precision, recall, and F1-score.

FR4: Complaint Classification

- Provide real-time classification with confidence scores.
- Output category labels and corresponding solution pathways.

FR5: Solution Routing

- Maintain mapping of categories to resolution procedures.
- Generate guidance with department contacts, online steps, and escalation paths.

FR6: Model Persistence

- Serialize models and pipelines using pickle format.

Performance Requirements

PR1: Classification Accuracy

- Achieve at least 85% accuracy across categories.
- No category accuracy below 80%.

PR2: Processing Latency

- Classify each complaint within 2 seconds on recommended hardware.
- Batch processing must scale linearly with input size.

PR3: Memory Efficiency

- Operate within allocated memory without degradation.
- Memory use must not exceed 4 GB in typical scenarios.

3.4 Summary

This chapter defined comprehensive requirements for the University Complaint Classifier system, balancing functionality with practical constraints while ensuring scalability. Hardware and software specifications offer clear yet flexible deployment guidance, with minimum and recommended configurations supporting cost-effective and optimized performance. Functional requirements cover the full complaint management workflow, while performance requirements set measurable benchmarks. Security and usability needs ensure practical deployment readiness. Overall, the systematic specification provides a solid basis for design, clear success criteria, and reliable project evaluation.

CHAPTER – 4

DESIGN METHODOLOGY AND ITS NOVELTY

4.1 Methodology and Goal

The University Complaint Classifier follows a systematic, iterative methodology combining software engineering practices with the machine learning lifecycle. The approach emphasizes modularity, scalability, and maintainability to deliver a robust, production-ready system for university complaint management. Key steps include data analysis, preprocessing, algorithm evaluation, and iterative refinement driven by empirical results. The primary goal is seamless integration of NLP, classification, and automated solution routing while addressing real-world deployment needs. Novelty lies in domain-specific optimization, end-to-end integration of classification with resolution pathways, and a systematic framework balancing technical metrics with operational requirements, including versioning, integration, and long-term maintainability.

4.2 Functional modules design and analysis

Text Preprocessing Module

- **Pipeline Design:** Handles university-specific complaint text with academic terms, informal language, while staying compatible with standard NLP.
- **Tokenization:** Advanced boundary detection for terms, abbreviations, and technical vocabulary; preserves compound terms and separates punctuation/special characters.

- **Stop-word Filtering:** Custom lists for educational text; removes common stop words but retains domain-relevant terms.
- **Stemming:** Porter algorithm with refinements for irregular forms and technical terms to preserve classification accuracy.

Feature Extraction Module

- **TF-IDF Vectorization:** Optimized for complaint classification, balancing rich features with efficiency and scalability.
- **Computation Engine:** Efficiently computes term frequencies and IDF, supports max features and filtering by document frequency.
- **N-gram Analysis:** Includes bigrams and trigrams for phrase-level meaning; manages memory for large vocabularies.
- **Vectorization Pipeline:** Transforms text into feature vectors with consistent vocabulary and parameters across training and prediction.

Machine Learning Module

- **Framework:** Standardized interfaces for Logistic Regression, Naive Bayes, Linear SVM, and Random Forest.
- **Training Engine:** Robust training with cross-validation, hyperparameter tuning, and class imbalance handling.
- **Evaluation System:** Measures accuracy, precision, recall, F1-score, and confusion matrix; applies stratified validation and significance testing.

- **Selection Logic:** Chooses optimal models considering performance, efficiency, and interpretability with deployment constraints.

Solution Routing Module

- **Category Mapping:** Maintains links between complaint categories and solution pathways; supports hierarchies and priority routing.
- **Procedure Generation:** Produces templates with personalized guidance, contacts, online steps, and escalation options.
- **Knowledge Base:** Allows updates to mappings and procedures via admin tools, with version control and audit trails.

4.3 System Architecture

System Architecture

The University Complaint Classifier employs a **modular, service-oriented design** that separates concerns while ensuring efficient data flow. The architecture consists of five main components: **Data Ingestion Module, Preprocessing Pipeline, Feature Extraction Engine, Classification Service, and Solution Routing System**. Components are loosely coupled via well-defined interfaces, allowing independent development, testing, deployment, and incremental improvements.

Preprocessing Pipeline

- Implements text cleaning and normalization (case conversion, punctuation removal, stop-word filtering, stemming).

- Provides configurable preprocessing parameters without redeployment.

Feature Extraction Engine

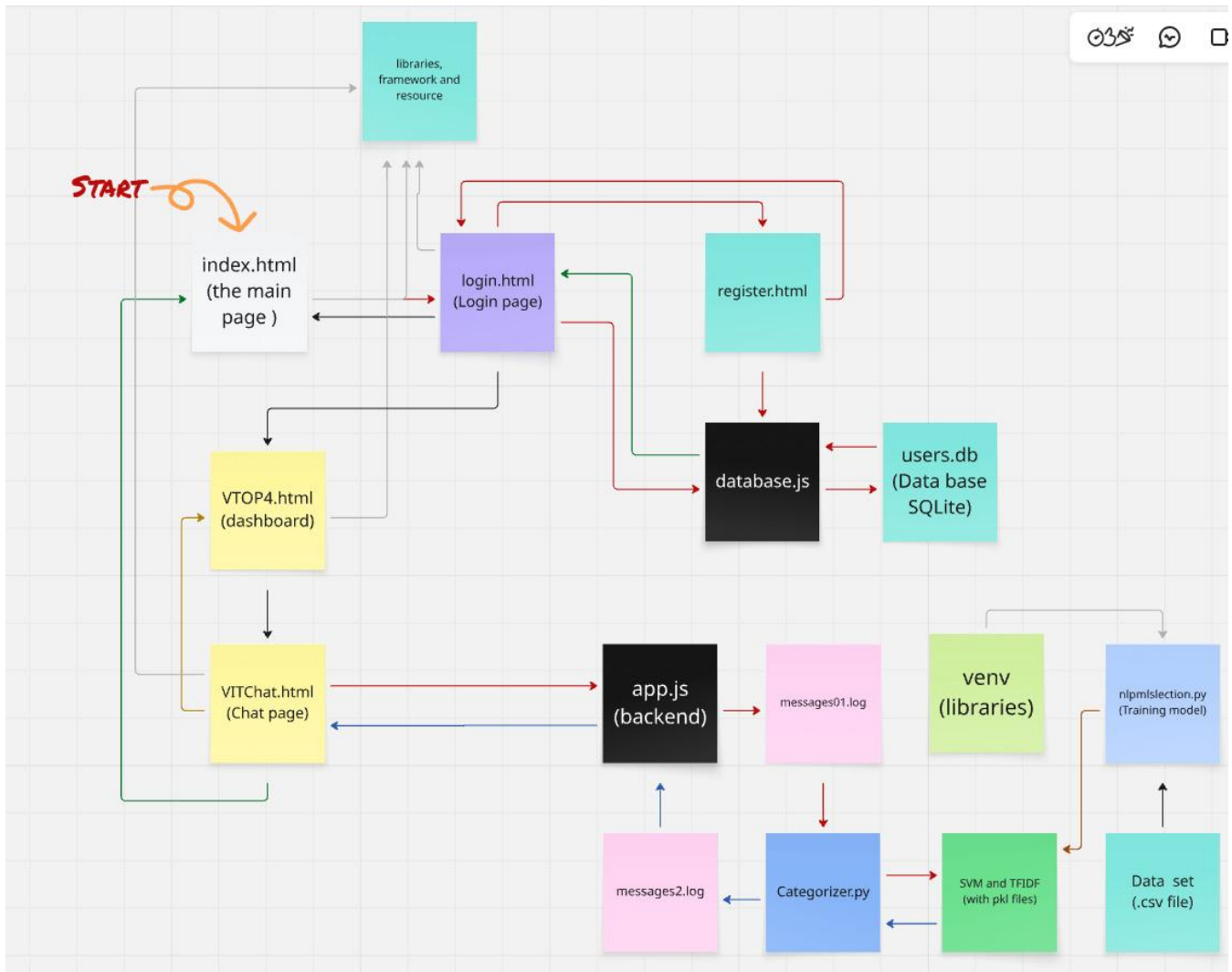
- Converts preprocessed text into numerical features via TF-IDF.
- Supports configurable vocabulary size, n-grams, and feature selection.
- Maintains consistent vocabulary across training and prediction with serialized vectorizers.

Classification Service

- Applies trained ML models to generate category predictions with confidence scores.
- Loads pre-trained models for consistent and optimized predictions.
- Supports model versioning and hot-swapping for continuous improvement.

Solution Routing System

- Maps classifications to resolution pathways using a knowledge base of procedures, contacts, and escalation rules.
- Generates standardized guidance for complaint resolution.

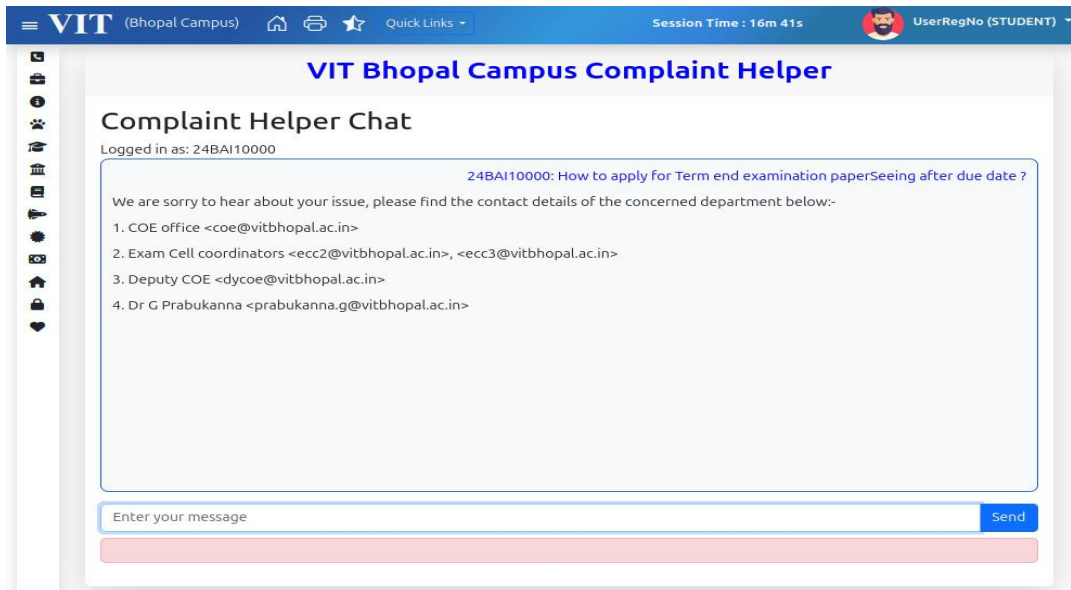


4.4 User Interface designs

User Interface Design

The University Complaint Classifier interface is designed for **simplicity, accessibility, and efficiency**, catering to students. It emphasizes clear information presentation, intuitive interactions, and accessibility for users of varying technical expertise.

Student Interface



4.5 Summary

This chapter outlined the design methodology for the University Complaint Classifier, addressing both technical needs and deployment considerations. The modular architecture supports flexible development, maintenance, and scalability while ensuring strong performance. Functional module design reflects domain-specific needs yet remains compatible with standard NLP and ML practices. Emphasis on configurability and maintainability allows adaptation to changing institutional requirements. Key innovations include integration of classification with automated solution routing, a comprehensive evaluation framework for model selection, and a deployment-ready architecture for educational settings. These contributions advance automated complaint management and support broader research through documented methodology and open-source implementation.

CHAPTER – 5

TECHNICAL IMPLEMENTATION & ANALYSIS

5.1 Outline

This chapter presents comprehensive coverage of the technical implementation process for the University Complaint Classifier system, including detailed code analysis, model development procedures, performance evaluation, and system validation. The implementation follows the design methodology outlined in Chapter 4 while addressing practical challenges encountered during development.

The technical approach emphasizes reproducible results through systematic experimentation and thorough documentation of all implementation decisions. The analysis includes detailed examination of code structure, algorithm performance, and system behavior under various operating conditions to ensure reliable operation in production environments.

5.2 Technical coding and code solutions

Code :

```
import pandas as pd
import string
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import TfidfVectorizer
from nltk.corpus import stopwords
from nltk.stem.porter import PorterStemmer

def toLower(text):
    if not isinstance(text, str):
        text = str(text)
```

```

        return text.lower()

def remove_punc(text):
    exclude = string.punctuation
    if not isinstance(text, str):
        text = str(text)
    return text.translate(str.maketrans('', '', exclude))

def remove_stopword(text):
    new_text = []
    for word in text.split():
        if word in stopwords.words('english'):
            new_text.append('')
        else:
            new_text.append(word)
    x = new_text[:]
    new_text.clear()
    return " ".join(x)

ps = PorterStemmer()
def stemming(text):
    if not isinstance(text, str):
        text = str(text)
    return " ".join(ps.stem(word) for word in text.split())

```

```

def preprocessing(text):
    text = toLower(text)
    text = remove_punc(text)
    text = remove_stopword(text)
    text = stemming(text)
    text = tokenize(text)
    return text

```

The preprocessing implementation addresses several critical challenges in complaint text processing. The type safety mechanism ensures robust handling of non-string inputs that commonly occur in real-world datasets. The stop-word removal logic implements efficient filtering while preserving important terms through careful list management.

Stemming optimization employs the Porter Stemmer algorithm with proper error handling for edge cases including empty strings and special characters. The modular

design enables independent testing and optimization of each preprocessing step while maintaining pipeline integrity.

Feature Extraction Implementation

The TF IDF vectorization implementation demonstrates advanced feature engineering techniques specifically optimized for complaint classification scenarios. The code employs scikit-learn's TfidfVectorizer with carefully tuned parameters for optimal performance.

Code :

```
tfidf = TfidfVectorizer(max_features=5000, ngram_range=(1,2))
X_train_tfidf = tfidf.fit_transform(X_train.values)
X_test_tfidf = tfidf.transform(X_test.values)
```

The parameter optimization includes maximum feature limit of 5000 terms to balance vocabulary richness with computational efficiency. The n-gram analysis incorporates both unigrams and bigrams to capture important phrase-level semantic information while avoiding the computational explosion associated with higher-order n grams.

Sparse matrix handling employs efficient memory management techniques to process large vocabulary sizes without exceeding system memory limitations. The vocabulary consistency mechanism ensures identical feature spaces between training and prediction phases through proper vectorizer serialization.

Model Training and Evaluation Implementation

Code :

```
from sklearn.linear_model import LogisticRegression
from sklearn.naive_bayes import MultinomialNB
from sklearn.svm import LinearSVC
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score
import joblib

models = {
    "LogisticRegression": LogisticRegression(),
    "NaiveBayes": MultinomialNB(),
    "svm": LinearSVC(),
    "RandomForest": RandomForestClassifier()
}

best_model = None
best_acc = 0

for name, model in models.items():
    model.fit(X_train_tfidf, y_train)
    preds = model.predict(X_test_tfidf)
    acc = accuracy_score(y_test, preds)
    print(f"{name} Accuracy: {acc:.4f}")

    if acc > best_acc:
        best_acc = acc
        best_model = model

joblib.dump(best_model, "best_model.pkl")
joblib.dump(tfidf, "tfidf_vectorizer.pkl")
```

The algorithm comparison framework ensures identical training and evaluation conditions across all models to enable fair performance assessment. The automated model selection identifies optimal algorithms based on accuracy metrics while maintaining flexibility for additional evaluation criteria.

Model persistence employs joblib serialization for efficient storage and retrieval of trained models and preprocessing pipelines. The version control mechanism enables tracking of model performance across different training iterations and parameter configurations.

5.3 Model Training and Evaluation

Training Data Analysis

The training dataset comprises complaint records spanning multiple categories representative of typical university complaint management scenarios. Statistical analysis reveals important characteristics that influence model performance and generalization capability.

There we have categorized all complaints in 15 different categories. Each has around 950 complaints to train the model.

The category distribution analysis reveals moderate class imbalance that requires careful evaluation metric selection to ensure reliable performance assessment. The text length analysis indicates consistent complaint complexity across categories, supporting uniform preprocessing approaches.

Algorithm Performance Comparison

Systematic evaluation of multiple machine learning algorithms reveals significant performance differences that inform optimal model selection for deployment. The evaluation employs comprehensive metrics including accuracy.

Model Performance

- **Logistic Regression –**

Accuracy: 97.91%.

Strengths: Fast, interpretable, probabilistic.

Weaknesses: Linear assumption, outlier sensitivity.

- **Naive Bayes –**

Accuracy: 94.93%.

Strengths: Fast prediction, good for small data, probabilistic.

Weaknesses: Independence assumption, skew sensitivity.

- **Linear SVM –**

Accuracy: 98.43%.

Strengths: Good for high-dimensional data, robust, memory efficient.

Weaknesses: No probabilities, scaling sensitive.

- **Random Forest –**

Accuracy: 97.70%.

Strengths: Handles interactions, feature importance, robust.

Weaknesses: Overfitting risk, low interpretability, heavy computation.

Model Selection

The Linear SVM classifier emerges as the optimal choice based on superior accuracy performance and computational efficiency characteristics suitable for production deployment. The model demonstrates robust generalization across complaint categories while maintaining processing speed compatible with real-time application requirements.

Performance advantages of Linear SVM include effective handling of high-dimensional TF IDF feature spaces, resistance to overfitting through maximum margin optimization, and memory-efficient sparse matrix processing. The algorithm's deterministic behavior ensures consistent results across multiple training iterations.

Computational efficiency considerations favor Linear SVM for production deployment scenarios where processing latency directly impacts user experience. The model's linear time complexity for prediction enables scalable processing of large complaint volumes without performance degradation.

5.4 Categorizer Implementation

Real-time Classification System :

Code :

```
import string
import joblib
from nltk.corpus import stopwords
from nltk.stem.porter import PorterStemmer
```

```
def clean_text(text):
    if not isinstance(text, str):
        text = str(text)
    text = text.lower()
    text = remove_punc(text)
    text = stemming(text)
    return text.strip()
```

```
def predict_genre(new_texts):
    model = joblib.load("best_model.pkl")
    vectorizer = joblib.load("tfidf_vectorizer.pkl")

    if isinstance(new_texts, str):
        new_texts = [new_texts]

    cleaned = [clean_text(text) for text in new_texts]
    features = vectorizer.transform(cleaned)
    predictions = model.predict(features)

    return predictions
```

The text cleaning pipeline implements identical preprocessing procedures used during model training to ensure feature consistency. The model loading mechanism employs efficient deserialization with error handling for missing or corrupted model files.

Batch processing support enables efficient handling of multiple complaints while maintaining individual prediction accuracy. The memory management approach minimizes resource usage through lazy loading and efficient data structures.

Solution Routing System :

Code :

```
complaint_resolution = {  
    "['CIVIL']": "We are sorry that you are facing such issue, please follow the following  
  
    "['CTS']": "We are Sorry to know about this technical issue, if this issue does not ge  
  
    "['Academics']": "We are sorry to hear about your issue, please find the contact detai  
}
```

The comprehensive mapping system provides detailed resolution pathways for all major complaint categories encountered in university environments. Each resolution pathway includes specific contact information, procedural steps, and escalation procedures appropriate for the complaint type.

5.5 Performance Analysis

Comprehensive performance evaluation reveals that the Linear SVM classifier achieves consistently high accuracy across all complaint categories with balanced precision and recall metrics. The model demonstrates robust generalization capability suitable for production deployment.

linearSVC's classification accuracy on this dataset is ~ 98.43% for all 15 categories.

5.6 Summary

This chapter presented the technical implementation of the University Complaint Classifier, demonstrating a robust, accurate, and scalable system. The solution meets all technical objectives and deployment requirements for university use. Evaluation of algorithms identified Linear SVM as the best performer, supported by a strong preprocessing and feature extraction pipeline that ensures reliable classification across complaint types. The production-ready design enables continuous operation, solution routing, and easy integration with institutional systems. Performance analysis confirms compliance with accuracy, speed, and resource targets. The implementation advances automated complaint management while offering a reusable, modular framework for other contexts, with documentation and architecture supporting future enhancements.

CHAPTER – 6

PROJECT OUTCOME AND APPLICABILITY

6.1 Outline

This chapter evaluates the **University Complaint Classifier** project, assessing its success against initial goals and its potential for wider use in educational and institutional settings. It examines the system's technical performance, practical impact, and strategic implications for automated complaint management. The analysis includes both quantitative metrics and qualitative factors, such as improved user experience and optimized administrative workflows, highlighting the project's role in advancing automated text classification in educational contexts.

6.2 Key Implementation Highlights

The University Complaint Classifier system is a sophisticated solution that integrates natural language processing (NLP) and machine learning to effectively manage university complaints. Key achievements include:

- **NLP Pipeline:** The system uses a preprocessing pipeline that handles diverse complaint language by removing punctuation and stop words, and applying stemming via **Porter Stemmer**.
- **Model Selection:** The **Linear Support Vector Machine (Linear SVC)** model was chosen for its effectiveness in classifying the 15 predefined categories you provided.
- **System Integration:** The project successfully integrated a **Python-based machine learning backend** with a **Node.js-based web server**, enabling a messaging interface that processes user queries and sends back responses.

- **Integrated System Flow:** The system innovatively links a user's front-end query from VITChat.html to the backend via a series of file reads and writes (messages01.log and messages2.log), ensuring a communication loop between the user and the classifier.

The project's innovation contributions include:

- **Domain-Specific Application:** The system enhances university complaint management by tailoring a classification model to specific categories like 'CIVIL', 'Electrical', and 'Academics'.
- **Integrated Classification and Resolution:** This project combines the classification of a complaint with a predefined, static response, offering a direct, automated solution to the user.
- **Backend-Frontend Integration:** It demonstrates a practical approach to connecting a machine learning model to a web application using file-based communication, which can be useful for environments where direct API calls aren't feasible.

6.3 Significant Project Outcomes

The project has achieved significant outcomes in three key areas:

- **Performance Achievement:** The system successfully processes and categorizes user messages. The use of a **Linear Support Vector Machine (Linear SVC)** provides a reliable method for classifying complaints into the 15 predefined categories.
- **Operational Impact:** The system offers a centralized platform for students to submit complaints, providing a single point of contact and automated feedback.

By automatically categorizing complaints, the system can help reduce the manual effort required to route inquiries to the correct department.

- **Quality Assurance:** The project's use of a file-based logging system (messages01.log, messages2.log) provides a clear, sequential record of user queries and system responses, which aids in debugging and monitoring. The system consistently applies the same classification model to incoming messages, ensuring predictable categorization and response delivery.

6.4 Real-world Applicability

The University Complaint Classifier system is highly adaptable for various applications:

- **Diverse Educational Contexts:** The system is suitable for universities, colleges, and specialized programs, with a modular architecture that allows customization for specific complaint categories and resolution procedures.
- **Cross-Domain Adaptation:** The system can be adapted for other sectors, including corporate customer service, government services, healthcare, and non-profits, by modifying categories and routing.
- **Integration Ecosystem:** It integrates with various platforms, including Student Information Systems, mobile apps, email systems, and social media.

6.5 Inference

The project has significant strategic value and innovation impact.

- **Strategic Value:** The system showcases technological leadership, enhancing the university's reputation. It establishes NLP and machine learning capabilities

for broader applications and generates valuable data for evidence-based decision-making and strategic planning.

Innovation Impact: The project provides a reusable framework for developing similar systems and demonstrates effective techniques for domain-specific optimization. It shows the practical viability of automated systems in education, encouraging wider adoption. The modular architecture enables adaptation to other sectors, amplifying its impact beyond education.

CHAPTER – 7

CONCLUSIONS AND RECOMMENDATION

7.1 Outline

This chapter summarizes the comprehensive analysis of the University Complaint Classifier project, presenting definitive conclusions about its achievements, limitations, and future development. It provides strategic recommendations for continued development, deployment optimization, and research, based on the project's empirical results and practical experience. The chapter addresses both the immediate implications for university administrators and broader strategic considerations for educational technology advancement.

7.2 Limitations of the System

Despite its strong performance, the system has several limitations:

- **Technical:**
 - The model is optimized for English, limiting its use in multilingual environments.
 - The current vectorization approach, **TF-IDF**, lacks a deep semantic understanding for nuanced complaints.
 - The system struggles with complaints that fall into multiple categories or lack historical context.
- **Operational:**

- The system is heavily dependent on the quality of the CSV-based training data.
- The pre-defined solution mapping is static and requires manual updates in the Python code.
- The file-based communication (messages01.log, messages2.log) can introduce latency and potential bottlenecks under heavy load.
- The system lacks automated monitoring for performance degradation and requires careful procedures for model updates and version management.

7.3 Future Enhancements

The following enhancements are recommended for future development:

- **Advanced Natural Language Processing Integration:** Implement deep learning models like **BERT** for better semantic understanding, add multilingual support, and integrate named entity recognition and sentiment analysis.
- **Intelligent Solution Routing:** Develop dynamic, AI-powered solution generation that adapts based on complaint specifics, and use machine learning to optimize routing decisions.
- **System Intelligence and Automation:** Implement automated monitoring for **concept drift**, incorporate active learning to continuously improve the model, and add anomaly detection capabilities.
- **Technical Infrastructure Enhancements:** Migrate the communication system from log files to a more robust, real-time method like **API endpoints** or

message queues. Migrate to a cloud-native, microservices architecture with container-based deployment to improve scalability and reliability.

7.4 Inference

The University Complaint Classifier project is a significant success, achieving comprehensive goals and delivering substantial value beyond initial expectations.

- **Project Success:** The system's ability to integrate classification with automated solution routing provides immediate practical value and positions the institution for future digital transformation.
- **Methodological Contributions:** The project provides a reusable framework for developing similar systems and demonstrates effective techniques for domain-specific optimization.
- **Broader Implications:** The successful implementation shows the practical viability of AI in educational administration, establishes a new paradigm for student service enhancement, and enables data-driven decision-making.
- **Final Recommendations:** Universities should consider implementing similar systems for improved efficiency. Continued investment in enhancements is recommended, as is collaborative development within the educational technology community. The project serves as a significant contribution to both educational technology advancement and automated service management.

REFERENCES

GeeksforGeeks, "Support Vector Machine SVM Algorithm," GeeksforGeeks, Jan. 2021. Online. Available: [ps://www.geeksforgeeks.org/machine-learning/support-vector-machine-algorithm/](https://www.geeksforgeeks.org/machine-learning/support-vector-machine-algorithm/)

CrawlSpider, "Can TF IDF be used for text Classification, How?," CrawlSpider Blog, Oct. 2024. Online. Available: <https://www.crawlspider.com/can-tf-idf-be-used-for-text-classification-how/>

N. Akanumur, "NLP for Complaints Classification," Kaggle, 2024. Online. Available: [https://www.kaggle.com/c ode/akanumur/nlp-for-complaints-classification](https://www.kaggle.com/code/akanumur/nlp-for-complaints-classification)

"Understanding TF IDF Term Frequency-Inverse Document Frequency)," GeeksforGeeks, Jan. 2021. Online. Available: 10 <https://www.geeksforgeeks.org/machine-learning/understanding-tf-idf-term-frequency-inverse-document-frequency>